

Tu es un développeur senior spécialisé en PHP 8.1, Symfony 6.1, Twig, Doctrine ORM, PostgreSQL, sécurité web et architecture d'applications de gestion.

Tu dois analyser, compléter et transformer le projet Symfony existant situé dans :

D:\SYMFONY\mon_projet

L'objectif est de créer une application web complète de gestion des paiements d'écolage pour une école, avec un back-office sécurisé pour l'administration et un front-office destiné principalement aux parents et à la consultation publique ou authentifiée.

1. Technologies déjà présentes et contraintes obligatoires

Le projet utilise déjà :

- PHP 8.1 ou supérieur ;
- Symfony 6.1 ;
- Twig ;
- Doctrine ORM ;
- Doctrine Migrations ;
- PostgreSQL ;
- Docker Compose ;
- PHPUnit ;
- HTML, CSS et JavaScript classique ;
- Symfony SecurityBundle ;
- Symfony Form ;
- Symfony Validator ;
- Symfony Mailer si disponible ;
- Symfony Monolog.

Tu dois conserver cette architecture.

Ne remplace pas Symfony par un autre framework.

Ne crée pas de frontend avec React, Vue, Angular, Vite ou Webpack.

Le rendu doit rester basé sur :

- Twig ;
- HTML5 ;
- CSS3 ;
- JavaScript natif ;
- éventuellement Bootstrap uniquement s'il est déjà présent ou s'il peut être ajouté proprement sans casser le projet.

Utilise Doctrine ORM pour toutes les interactions avec PostgreSQL.

Utilise les migrations Doctrine pour créer ou modifier la base de données.

Respecte les bonnes pratiques Symfony 6.1 et PHP 8.1.

2. Méthode de travail obligatoire

Avant de modifier le projet :

1. Analyse toute l'architecture existante.
2. Lis notamment :
3. `composer.json` ;
4. `config/bundles.php` ;
5. `config/packages/` ;
6. `config/routes/` ;
7. `src/Controller/` ;
8. `src/Entity/` ;
9. `src/Repository/` ;
10. `src/Form/` ;
11. `src/Security/` ;
12. `templates/` ;
13. `public/` ;
14. `migrations/` ;
15. `tests/` ;
16. `compose.yaml` ;
17. `.env` et `.env.example` s'ils existent.
18. Identifie les fonctionnalités déjà présentes.
19. Réutilise les contrôleurs, services, templates, composants Twig, styles et logiques existants autant que possible.
20. Ne recrée pas inutilement une fonctionnalité déjà présente.
21. Ne supprime pas une fonctionnalité existante sauf si elle est clairement inutilisée ou incorrecte.
22. Corrige les éventuels problèmes de configuration avant de commencer les nouvelles fonctionnalités.
23. Travaille directement dans les fichiers du projet.
24. À la fin, exécute les commandes de validation, les migrations et les tests disponibles.
25. Corrige les erreurs rencontrées avant de considérer le projet comme terminé.

3. Objectif général

Créer une application appelée provisoirement :

SchoolPay

L'application doit permettre à une école de :

- gérer les années scolaires ;
- gérer les utilisateurs et leurs rôles ;

- gérer les classes ;
- gérer les élèves ;
- gérer les parents ou responsables légaux ;
- définir les différents frais scolaires ;
- enregistrer les paiements ;
- accepter les paiements partiels ;
- appliquer des réductions ;
- produire des reçus ;
- suivre les impayés ;
- générer des rapports ;
- afficher des statistiques ;
- conserver un journal d'audit ;
- permettre aux parents de consulter la situation financière de leurs enfants.

L'application doit comporter :

- un back-office ;
- un front-office ;
- une authentification ;
- une gestion fine des droits ;
- une interface responsive pour ordinateur, tablette et téléphone.

4. Utilisateurs et rôles

Créer les rôles Symfony suivants :

- ROLE_ADMIN ;
- ROLE_COMPTABLE ;
- ROLE_DIRECTEUR ;
- ROLE_PARENT.

ROLE_ADMIN

L'administrateur peut :

- accéder à tout le back-office ;
- gérer les utilisateurs ;
- gérer les rôles ;
- gérer les années scolaires ;
- gérer les classes ;
- gérer les élèves ;
- gérer les parents ;
- gérer les frais scolaires ;
- gérer les réductions ;
- consulter tous les paiements ;
- annuler un paiement ;
- consulter tous les rapports ;
- consulter les journaux d'audit ;
- modifier les paramètres de l'école ;
- lancer les sauvegardes ou exports disponibles.

ROLE_COMPTABLE

Le comptable peut :

- consulter les élèves ;
- rechercher un élève ;
- enregistrer un paiement ;
- enregistrer plusieurs versements ;
- imprimer ou télécharger un reçu ;
- consulter l'historique des paiements ;
- consulter les impayés ;
- appliquer une réduction uniquement selon les permissions prévues ;
- annuler un paiement avec justification ;
- consulter les rapports financiers autorisés.

ROLE_DIRECTEUR

Le directeur peut principalement consulter :

- le tableau de bord ;
- les montants encaissés ;
- les montants attendus ;
- les impayés ;
- les rapports ;
- les statistiques par classe, niveau, période et mode de paiement.

Il ne doit pas pouvoir modifier les paiements sauf si une permission spécifique lui est accordée.

ROLE_PARENT

Le parent peut :

- se connecter au front-office ;
- voir uniquement ses enfants ;
- consulter les frais affectés à ses enfants ;
- voir les paiements effectués ;
- voir le reste à payer ;
- télécharger les reçus ;
- consulter les échéances ;
- recevoir ou consulter les notifications ;
- modifier certaines informations personnelles limitées.

5. Authentification et sécurité

Créer ou compléter le système d'authentification Symfony.

Fonctionnalités attendues :

- page de connexion ;

- déconnexion ;
- mot de passe hashé ;
- gestion des rôles ;
- contrôle d'accès par route ;
- protection CSRF ;
- validation serveur ;
- protection contre les injections SQL grâce à Doctrine ;
- expiration de session ;
- messages d'erreur propres ;
- redirection selon le rôle après connexion ;
- impossibilité pour un parent d'accéder aux données d'un autre parent ;
- impossibilité pour un utilisateur non autorisé d'accéder au back-office.

Prévoir une page d'accès refusé propre.

Créer des fixtures de démonstration avec au minimum :

- un administrateur ;
- un comptable ;
- un directeur ;
- un parent.

Documenter les identifiants de démonstration dans le README, uniquement pour l'environnement de développement.

6. Structure générale des interfaces

Back-office

Prévoir une mise en page avec :

- barre latérale ;
- barre supérieure ;
- menu utilisateur ;
- fil d'Ariane ;
- notifications ;
- contenu principal ;
- pied de page ;
- design responsive ;
- confirmation avant les actions sensibles ;
- messages flash de succès ou d'erreur.

Le menu du back-office doit contenir :

- Tableau de bord ;
- Années scolaires ;
- Classes ;
- Élèves ;
- Parents ;
- Frais scolaires ;

- Paiements ;
- Réductions ;
- Impayés ;
- Reçus ;
- Rapports ;
- Utilisateurs ;
- Journal d'audit ;
- Paramètres.

Les éléments du menu doivent s'afficher selon le rôle connecté.

Front-office

Créer un front-office moderne comprenant au minimum :

- page d'accueil ;
- page de connexion parent ;
- espace parent ;
- liste des enfants du parent ;
- détail financier d'un enfant ;
- historique des paiements ;
- reçus téléchargeables ;
- échéances ;
- notifications ;
- profil du parent ;
- page de contact ou d'informations de l'école.

7. Entités Doctrine à créer

Créer les entités nécessaires avec leurs relations, validations, contraintes et index.

Tu peux adapter les noms selon les conventions du projet, mais le modèle fonctionnel doit couvrir les éléments suivants.

User

Champs recommandés :

- id ;
- email unique ;
- roles ;
- password ;
- firstName ;
- lastName ;
- phone ;
- isActive ;
- createdAt ;
- updatedAt ;
- lastLoginAt.

Relations possibles :

- parentProfile ;
- paiements enregistrés ;
- actions d'audit.

SchoolYear

Champs :

- id ;
- name, par exemple 2026-2027 ;
- startDate ;
- endDate ;
- isActive ;
- isClosed ;
- createdAt ;
- updatedAt.

Règles :

- une seule année scolaire active ;
- une année clôturée ne peut plus être modifiée librement ;
- les paiements restent associés à l'année concernée.

SchoolClass

Champs :

- id ;
- name ;
- level ;
- description ;
- capacity ;
- isActive ;
- schoolYear ;
- createdAt ;
- updatedAt.

ParentGuardian

Champs :

- id ;
- firstName ;
- lastName ;
- email ;
- phone ;
- alternatePhone ;
- address ;

- profession ;
- relationshipType ;
- user associé ;
- createdAt ;
- updatedAt.

Un parent peut avoir plusieurs enfants.

Un enfant peut éventuellement avoir plusieurs responsables légaux.

Prévoir une relation adaptée, de préférence ManyToMany avec une entité intermédiaire si des informations supplémentaires sont nécessaires.

Student

Champs :

- id ;
- registrationNumber unique ;
- firstName ;
- lastName ;
- gender ;
- birthDate ;
- birthPlace ;
- address ;
- photo ;
- status ;
- enrollmentDate ;
- schoolClass ;
- schoolYear ;
- createdAt ;
- updatedAt.

Ajouter :

- recherche multicritère ;
- filtre par classe ;
- filtre par statut ;
- filtre par sexe ;
- filtre par année scolaire ;
- pagination.

FeeType

Représente un type de frais.

Exemples :

- inscription ;
- écolage ;
- cantine ;

- transport ;
- uniforme ;
- bibliothèque ;
- examen ;
- assurance ;
- autre.

Champs :

- id ;
- name ;
- code unique ;
- description ;
- isMandatory ;
- isRecurring ;
- recurrenceType ;
- isActive ;
- createdAt ;
- updatedAt.

FeeAssignment

Représente un frais défini pour une classe, un élève ou une année scolaire.

Champs :

- id ;
- feeType ;
- schoolYear ;
- schoolClass facultative ;
- student facultatif ;
- amount ;
- dueDate ;
- startDate ;
- endDate ;
- monthNumber ou période si nécessaire ;
- isMandatory ;
- description ;
- createdAt ;
- updatedAt.

Le système doit pouvoir définir :

- un frais pour toute une classe ;
- un frais individuel pour un élève ;
- un frais mensuel ;
- un frais unique ;
- un frais obligatoire ;
- un frais facultatif.

Discount

Champs :

- id ;
- name ;
- type : montant fixe ou pourcentage ;
- value ;
- reason ;
- startDate ;
- endDate ;
- isActive ;
- createdAt ;
- updatedAt.

StudentDiscount

Associe une réduction à un élève ou à un frais précis.

Champs :

- id ;
- student ;
- discount ;
- feeAssignment facultatif ;
- schoolYear ;
- approvedBy ;
- justification ;
- createdAt.

Types de réductions à prévoir :

- bourse ;
- fratrie ;
- enfant d'employé ;
- exonération ;
- réduction exceptionnelle ;
- remise fixe ;
- remise en pourcentage.

Payment

Champs :

- id ;
- paymentNumber unique ;
- student ;
- schoolYear ;
- paymentDate ;
- totalAmount ;

- paymentMethod ;
- externalReference ;
- status ;
- notes ;
- receivedBy ;
- cancellationReason ;
- cancelledAt ;
- cancelledBy ;
- createdAt ;
- updatedAt.

Modes de paiement :

- espèces ;
- MVola ;
- Orange Money ;
- Airtel Money ;
- virement bancaire ;
- carte bancaire ;
- chèque ;
- autre.

Statuts possibles :

- validé ;
- partiellement affecté ;
- annulé ;
- remboursé si cette option est ajoutée.

PaymentDetail

Permet de répartir un paiement sur plusieurs frais.

Champs :

- id ;
- payment ;
- feeAssignment ;
- amount ;
- createdAt.

Un paiement peut couvrir plusieurs frais.

Un frais peut être payé en plusieurs fois.

Receipt

Champs :

- id ;
- receiptNumber unique ;

- payment ;
- generatedAt ;
- filePath facultatif ;
- verificationCode unique ;
- qrCodeData ;
- createdAt.

Un reçu ne doit jamais être supprimé physiquement.

Notification

Champs :

- id ;
- user ou parent ;
- title ;
- message ;
- type ;
- isRead ;
- sentAt ;
- createdAt.

AuditLog

Champs :

- id ;
- user ;
- action ;
- entityType ;
- entityId ;
- description ;
- oldValues au format JSON ;
- newValues au format JSON ;
- ipAddress ;
- userAgent ;
- createdAt.

SchoolSetting

Champs possibles :

- schoolName ;
- schoolCode ;
- address ;
- phone ;
- email ;
- logo ;
- currency ;
- receiptFooter ;

- defaultPaymentDeadline ;
- lateFeeEnabled ;
- createdAt ;
- updatedAt.

8. Gestion des années scolaires

Créer les pages permettant de :

- lister les années scolaires ;
- créer une année scolaire ;
- modifier une année scolaire ;
- activer une année scolaire ;
- clôturer une année scolaire ;
- consulter une année clôturée ;
- empêcher les modifications interdites après clôture.

Lorsqu'une nouvelle année devient active, l'ancienne doit être désactivée automatiquement.

9. Gestion des classes

Créer les fonctionnalités :

- liste des classes ;
- création ;
- modification ;
- activation ou désactivation ;
- détail d'une classe ;
- liste des élèves de la classe ;
- frais associés à la classe ;
- statistiques de paiement de la classe.

Interdire la suppression d'une classe contenant des élèves ou des données financières.

Privilégier une désactivation logique au lieu d'une suppression physique.

10. Gestion des élèves

Créer :

- liste paginée ;
- formulaire de création ;
- formulaire de modification ;
- fiche détaillée ;
- photo facultative ;
- affectation à une classe ;
- association avec un ou plusieurs parents ;
- historique scolaire ;
- situation financière ;

- historique des paiements ;
- reçus ;
- frais en attente ;
- réductions ;
- recherche multicritère.

Prévoir un import CSV.

L'import doit :

- afficher les colonnes attendues ;
- vérifier les données ;
- signaler les erreurs ligne par ligne ;
- éviter les doublons de matricule ;
- produire un résumé de l'import ;
- enregistrer les données valides ;
- permettre de télécharger un modèle CSV.

Prévoir également un export CSV de la liste des élèves.

11. Gestion des parents

Créer :

- liste des parents ;
- ajout ;
- modification ;
- détail ;
- association avec les enfants ;
- création facultative d'un compte utilisateur ;
- activation ou désactivation de l'accès ;
- consultation de la situation financière globale de tous les enfants du parent.

Un parent ne doit jamais pouvoir consulter les enfants qui ne lui sont pas associés.

12. Gestion des frais scolaires

Créer une interface permettant de :

- gérer les types de frais ;
- définir un frais pour une année scolaire ;
- affecter un frais à une classe ;
- affecter un frais à un élève ;
- définir un montant ;
- définir une date limite ;
- indiquer si le frais est obligatoire ;
- indiquer s'il est unique ou récurrent ;
- gérer l'écolage mensuel ;
- afficher les frais échus ;
- afficher les frais à venir.

Pour l'écolage mensuel, permettre de générer automatiquement les échéances pour plusieurs mois.

Exemple :

- septembre : 100 000 Ar ;
- octobre : 100 000 Ar ;
- novembre : 100 000 Ar.

13. Gestion des paiements

Créer un module de caisse complet.

Le comptable doit pouvoir :

1. rechercher un élève par :
2. matricule ;
3. nom ;
4. prénom ;
5. classe ;
6. téléphone du parent ;
7. ouvrir la fiche de paiement de l'élève ;
8. voir :
9. les frais dus ;
10. les réductions ;
11. les montants déjà payés ;
12. le reste à payer ;
13. les retards ;
14. sélectionner un ou plusieurs frais ;
15. saisir le montant reçu ;
16. choisir le mode de paiement ;
17. renseigner une référence ;
18. ajouter une observation ;
19. valider le paiement ;
20. générer un reçu.

Le système doit gérer :

- paiement total ;
- paiement partiel ;
- paiement de plusieurs frais en une seule opération ;
- paiement anticipé ;
- paiement en retard ;
- trop-perçu ;
- ordre d'affectation des montants ;
- solde restant ;

- annulation contrôlée.

Le montant d'un paiement doit être strictement positif.

Le système doit utiliser des transactions Doctrine pour garantir l'intégrité des paiements.

Un paiement validé ne doit pas être supprimé.

En cas d'erreur, il doit être annulé avec :

- motif obligatoire ;
- date ;
- utilisateur ayant annulé ;
- trace dans le journal d'audit.

14. Calculs financiers

Créer un service métier dédié, par exemple :

`src/Service/PaymentCalculationService.php`

Ce service doit calculer :

- montant brut des frais ;
- total des réductions ;
- montant net attendu ;
- montant déjà payé ;
- reste à payer ;
- éventuel trop-perçu ;
- statut du frais ;
- statut financier de l'élève.

Statuts possibles d'un frais :

- non payé ;
- partiellement payé ;
- payé ;
- en retard ;
- exonéré ;
- annulé si applicable.

Éviter de placer les règles financières directement dans les contrôleurs.

Les contrôleurs doivent rester légers.

15. Réductions et exonérations

Créer les pages pour :

- définir une réduction ;
- modifier une réduction ;
- associer une réduction à un élève ;
- associer une réduction à un frais ;
- approuver une réduction ;
- saisir une justification ;
- consulter l'historique des réductions.

Le système doit accepter :

- montant fixe ;
- pourcentage.

Une réduction ne doit jamais rendre le montant à payer négatif.

Les modifications doivent être enregistrées dans l'audit.

16. Reçus

Après validation d'un paiement, générer un reçu contenant :

- logo de l'école ;
- nom de l'école ;
- adresse ;
- téléphone ;
- email ;
- numéro unique du reçu ;
- date et heure ;
- matricule de l'élève ;
- nom complet de l'élève ;
- classe ;
- année scolaire ;
- détail des frais payés ;
- montant total ;
- montant en lettres si possible ;
- mode de paiement ;
- référence de transaction ;
- nom du comptable ;
- reste à payer ;
- QR Code ou code de vérification ;
- pied de page configuré.

Le reçu doit être :

- imprimable depuis Twig ;
- téléchargeable en PDF si une bibliothèque PDF compatible est disponible ou ajoutée ;

- consultable dans le back-office ;
- consultable dans l'espace parent ;
- impossible à supprimer.

Créer une page publique ou semi-publique de vérification du reçu à partir du code unique ou du QR Code.

Cette page ne doit afficher que les informations minimales nécessaires à la vérification.

17. Impayés

Créer un module permettant de consulter :

- les élèves avec un reste à payer ;
- les frais échus ;
- le montant total restant ;
- le nombre de jours de retard ;
- la classe ;
- le parent responsable ;
- le téléphone ;
- l'email ;
- la dernière date de paiement.

Ajouter des filtres :

- année scolaire ;
- classe ;
- niveau ;
- type de frais ;
- date d'échéance ;
- montant minimum ;
- parent ;
- statut.

Prévoir :

- export CSV ;
- impression ;
- sélection des dossiers à relancer ;
- création d'une notification interne.

18. Tableau de bord

Créer un tableau de bord adapté à chaque rôle.

Tableau de bord administrateur

Afficher :

- nombre total d'élèves ;
- nombre de parents ;
- nombre de classes ;
- nombre d'utilisateurs ;
- montant total attendu ;
- montant total encaissé ;
- montant restant ;
- taux de recouvrement ;
- paiements du jour ;
- paiements du mois ;
- nombre d'impayés ;
- dernières opérations ;
- alertes ;
- activité récente.

Tableau de bord comptable

Afficher :

- paiements du jour ;
- montant encaissé aujourd'hui ;
- paiements récents ;
- raccourci vers un nouveau paiement ;
- élèves en retard ;
- reçus récents ;
- répartition par mode de paiement.

Tableau de bord directeur

Afficher :

- recettes par jour ;
- recettes par mois ;
- recettes par classe ;
- recettes par niveau ;
- évolution du taux de recouvrement ;
- montant des impayés ;
- principales alertes.

Tableau de bord parent

Afficher :

- enfants associés ;
- montant total attendu ;
- montant payé ;

- reste à payer ;
- prochaines échéances ;
- derniers paiements ;
- reçus récents ;
- notifications non lues.

Les graphiques peuvent être réalisés avec Chart.js chargé simplement dans les templates Twig.

19. Rapports

Créer un module de rapports avec filtres par période.

Rapports attendus :

- rapport journalier des paiements ;
- rapport hebdomadaire ;
- rapport mensuel ;
- rapport annuel ;
- rapport par classe ;
- rapport par niveau ;
- rapport par élève ;
- rapport par parent ;
- rapport par type de frais ;
- rapport par mode de paiement ;
- rapport par comptable ;
- rapport des réductions ;
- rapport des annulations ;
- rapport des impayés ;
- rapport du taux de recouvrement.

Chaque rapport doit proposer selon le cas :

- affichage HTML ;
- impression ;
- export CSV ;
- export PDF si possible.

20. Notifications

Créer au minimum un système de notifications internes.

Types de notifications :

- nouveau paiement ;
- paiement annulé ;
- échéance proche ;
- échéance dépassée ;
- reçu disponible ;
- réduction accordée ;
- compte parent créé.

Préparer l'architecture pour permettre plus tard :

- email ;
- SMS ;
- WhatsApp ;
- MVola ;
- Orange Money ;
- Airtel Money.

Ne pas simuler une intégration réelle avec ces services sans configuration ou API officielle.

Créer plutôt des interfaces ou services extensibles.

21. Journal d'audit

Créer un système centralisé d'audit.

Enregistrer au minimum :

- connexion ;
- échec de connexion important si pertinent ;
- création ;
- modification ;
- désactivation ;
- paiement ;
- annulation de paiement ;
- réduction ;
- clôture d'année scolaire ;
- changement de paramètres ;
- génération ou consultation sensible de rapports.

Créer une page réservée à l'administrateur avec :

- pagination ;
- filtres par utilisateur ;
- filtres par action ;
- filtres par entité ;
- filtres par période ;
- détail des anciennes et nouvelles valeurs.

22. Paramètres de l'école

Créer une page permettant de configurer :

- nom de l'école ;
- logo ;
- adresse ;
- téléphone ;
- email ;

- devise, par défaut Ariary ;
- symbole monétaire, Ar ;
- pied de page des reçus ;
- année scolaire active ;
- format des numéros de paiement ;
- format des numéros de reçu ;
- délai des échéances ;
- options de retard.

Les paramètres doivent être réutilisés dans les templates et les reçus.

23. Suppression et archivage

Pour les données sensibles, privilégier :

- désactivation ;
- archivage ;
- annulation ;
- suppression logique.

Ne jamais supprimer physiquement :

- un paiement validé ;
- un reçu ;
- un journal d'audit ;
- une donnée financière déjà utilisée.

Les classes, élèves, parents, frais et utilisateurs utilisés dans des transactions doivent être désactivés ou archivés au lieu d'être supprimés.

24. Recherche, filtres et pagination

Toutes les grandes listes doivent avoir :

- recherche ;
- filtres ;
- tri ;
- pagination ;
- conservation des filtres dans l'URL ;
- remise à zéro des filtres.

Cela concerne notamment :

- utilisateurs ;
- élèves ;
- parents ;
- classes ;
- frais ;
- paiements ;
- reçus ;

- impayés ;
- audits ;
- rapports.

25. Validation des formulaires

Utiliser :

- Symfony Form ;
- Symfony Validator ;
- contraintes Doctrine ;
- messages de validation en français.

Valider notamment :

- unicité du matricule ;
- unicité de l'email utilisateur ;
- unicité des numéros de paiement et de reçu ;
- montants positifs ;
- dates cohérentes ;
- année scolaire valide ;
- classe valide ;
- frais valide ;
- parent valide ;
- format des numéros de téléphone ;
- fichiers image autorisés ;
- taille maximale des fichiers.

26. Design et expérience utilisateur

Créer un design moderne, professionnel et cohérent.

Prévoir :

- couleurs adaptées à une école ;
- cartes statistiques ;
- tableaux lisibles ;
- badges de statut ;
- formulaires bien organisés ;
- boutons cohérents ;
- fenêtres de confirmation ;
- messages flash ;
- état vide lorsqu'aucune donnée n'est disponible ;
- indicateurs de chargement en JavaScript si nécessaire ;
- responsive mobile ;
- menu repliable ;
- impression propre des reçus et rapports.

Créer des templates Twig réutilisables :

- layout du back-office ;
- layout du front-office ;
- sidebar ;
- navbar ;
- breadcrumbs ;
- messages flash ;
- pagination ;
- formulaires ;
- modales de confirmation ;
- composants de statistiques ;
- badges de statut.

27. Services métier recommandés

Créer des services séparés lorsque nécessaire, notamment :

- PaymentCalculationService ;
- PaymentService ;
- ReceiptService ;
- ReceiptNumberGenerator ;
- PaymentNumberGenerator ;
- FeeAssignmentService ;
- OutstandingBalanceService ;
- ReportService ;
- AuditLoggerService ;
- NotificationService ;
- SchoolYearService ;
- CsvImportService ;
- CsvExportService.

Éviter les contrôleurs trop longs.

Éviter de dupliquer les calculs dans plusieurs fichiers.

28. Contrôleurs recommandés

Créer ou compléter des contrôleurs organisés, par exemple :

src/Controller/Backoffice/DashboardController.php src/Controller/Backoffice/SchoolYearController.php
src/Controller/Backoffice/SchoolClassController.php src/Controller/Backoffice/StudentController.php src/
Controller/Backoffice/ParentGuardianController.php src/Controller/Backoffice/FeeTypeController.php
src/Controller/Backoffice/FeeAssignmentController.php src/Controller/Backoffice/
PaymentController.php src/Controller/Backoffice/ReceiptController.php src/Controller/Backoffice/
DiscountController.php src/Controller/Backoffice/OutstandingPaymentController.php src/Controller/
Backoffice/ReportController.php src/Controller/Backoffice/UserController.php src/Controller/Backoffice/
AuditLogController.php src/Controller/Backoffice/SettingController.php

src/Controller/Frontoffice/HomeController.php src/Controller/Frontoffice/
ParentDashboardController.php src/Controller/Frontoffice/ChildController.php src/Controller/
Frontoffice/ParentReceiptController.php src/Controller/Frontoffice/ParentNotificationController.php src/
Controller/Frontoffice/ProfileController.php src/Controller/ReceiptVerificationController.php

Adapte cette structure à l'existant si le projet possède déjà une organisation différente.

29. Routes

Utiliser des noms de routes explicites.

Exemples :

- app_login ;
- app_logout ;
- backoffice_dashboard ;
- backoffice_student_index ;
- backoffice_student_new ;
- backoffice_student_edit ;
- backoffice_student_show ;
- backoffice_payment_new ;
- backoffice_payment_show ;
- backoffice_payment_cancel ;
- backoffice_receipt_print ;
- backoffice_report_monthly ;
- parent_dashboard ;
- parent_child_show ;
- parent_receipt_download ;
- receipt_verify.

Préfixes suggérés :

- /admin ou /backoffice ;
- /parent ;
- /receipt/verify.

30. Fixtures

Créer des fixtures cohérentes pour tester l'application.

Ajouter au minimum :

- une école configurée ;
- une année scolaire active ;
- plusieurs classes ;
- plusieurs types de frais ;
- plusieurs élèves ;
- plusieurs parents ;
- des relations parents-enfants ;

- des frais affectés ;
- des réductions ;
- des paiements partiels ;
- des paiements complets ;
- des impayés ;
- plusieurs utilisateurs avec rôles différents ;
- des notifications ;
- des journaux d'audit de démonstration.

Ne jamais utiliser les fixtures en production.

31. Tests

Créer des tests PHPUnit pour les éléments critiques.

Tests minimums :

- authentification ;
- accès au back-office selon le rôle ;
- interdiction d'accès pour un parent ;
- calcul du montant attendu ;
- calcul des réductions ;
- calcul du reste à payer ;
- paiement partiel ;
- paiement total ;
- impossibilité d'un montant négatif ;
- annulation de paiement ;
- génération des numéros uniques ;
- impossibilité pour un parent d'accéder à un enfant d'un autre parent ;
- clôture d'une année scolaire ;
- création d'un reçu ;
- recherche d'un élève.

Créer des tests unitaires pour les services métier et des tests fonctionnels pour les routes principales.

32. Base de données et migrations

Créer toutes les migrations Doctrine nécessaires.

Ajouter :

- clés étrangères ;
- contraintes d'unicité ;
- index sur les colonnes de recherche ;
- index sur les dates de paiement ;
- index sur les matricules ;
- index sur les statuts ;
- index sur l'année scolaire ;
- index sur les classes ;

- index sur les références financières.

Vérifier la compatibilité avec PostgreSQL.

Ne pas écrire de SQL spécifique à MySQL.

33. Commandes et installation

À la fin, vérifier que le projet peut être lancé avec une procédure claire.

Mettre à jour le README.md avec :

- prérequis ;
- installation des dépendances ;
- configuration du fichier .env.local ;
- lancement de PostgreSQL avec Docker Compose ;
- création de la base ;
- exécution des migrations ;
- chargement des fixtures ;
- lancement du serveur Symfony ;
- exécution des tests ;
- comptes de démonstration ;
- structure des rôles ;
- principales URLs.

Commandes indicatives à vérifier et adapter :

```
composer install docker compose up -d php bin/console doctrine:database:create --if-not-exists php
bin/console doctrine:migrations:migrate --no-interaction php bin/console doctrine:fixtures:load --no-
interaction php bin/console cache:clear php bin/console lint:container php bin/console lint:twig
templates php bin/console doctrine:schema:validate php bin/phpunit
```

Ne lance pas une commande destructive sur une base contenant des données réelles.

34. Qualité du code

Respecter :

- PSR-12 ;
- typage PHP ;
- propriétés typées ;
- types de retour ;
- injection de dépendances ;
- contrôleurs légers ;
- services métier ;
- repositories dédiés ;
- formulaires Symfony ;
- validateurs ;
- voter Symfony lorsque nécessaire ;

- transactions Doctrine pour les paiements ;
- messages en français ;
- noms de classes et méthodes cohérents ;
- documentation des parties complexes.

Ne pas mettre la logique métier dans Twig.

Ne pas faire de requêtes SQL directement dans les templates.

Ne pas utiliser de variables globales inutiles.

Ne pas stocker les mots de passe en clair.

35. Priorités d'implémentation

Implémente le projet dans cet ordre :

1. analyse et correction de l'existant ;
2. authentification et rôles ;
3. entités et migrations ;
4. fixtures ;
5. layout back-office et front-office ;
6. années scolaires ;
7. classes ;
8. élèves ;
9. parents ;
10. types de frais et affectations ;
11. réductions ;
12. paiements et calculs ;
13. reçus ;
14. impayés ;
15. tableau de bord ;
16. rapports ;
17. notifications ;
18. journal d'audit ;
19. paramètres ;
20. espace parent ;
21. import et export CSV ;
22. tests ;
23. documentation ;
24. validation finale.

36. Résultat final attendu

Le projet doit être réellement fonctionnel.

Il ne faut pas seulement créer :

- des pages vides ;

- des contrôleurs avec TODO ;
- des boutons sans action ;
- des formulaires non enregistrés ;
- des données simulées directement dans Twig ;
- des statistiques codées en dur.

Toutes les fonctionnalités doivent utiliser la base PostgreSQL.

Les formulaires doivent enregistrer réellement les données.

Les tableaux de bord doivent utiliser de vraies requêtes Doctrine.

Les paiements doivent mettre à jour correctement la situation financière.

Les reçus doivent correspondre aux paiements enregistrés.

Les droits doivent être réellement contrôlés côté serveur.

37. Rapport final obligatoire

Après avoir terminé, affiche un rapport contenant :

1. résumé de l'architecture analysée ;
2. fonctionnalités déjà présentes avant modification ;
3. fonctionnalités ajoutées ;
4. liste des fichiers créés ;
5. liste des fichiers modifiés ;
6. liste des migrations créées ;
7. entités et relations ajoutées ;
8. routes principales ;
9. rôles et autorisations ;
10. comptes de démonstration ;
11. commandes exécutées ;
12. tests exécutés ;
13. erreurs rencontrées et corrections appliquées ;
14. fonctionnalités qui nécessitent une API externe ;
15. procédure exacte pour lancer le projet.

Ne prétends pas qu'une commande ou un test a réussi si tu ne l'as pas réellement exécuté.

Lorsque certaines fonctionnalités externes ne peuvent pas être activées, comme les paiements MVola ou Orange Money, crée uniquement une architecture prête pour l'intégration et explique clairement ce qui reste à configurer.

Commence maintenant par analyser intégralement le projet existant, puis réalise toutes les modifications nécessaires directement dans le projet.